Azure Monitor and Alerts Practical Beginner Guide

Monitoring allows you to fix system issues before your end users even notice something went wrong. Learning how to collect logs, set alert thresholds, and read telemetry gives you the superpower to keep production environments reliable.

What you'll learn:

1. Tracking Live Resource Metrics
2. Querying Crash Logs with KQL
3. Creating Automated Threshold Alerts
4. Auditing Changes via Activity Log

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Tracking Live Resource Metrics

Every cloud resource emits built-in numerical metrics every minute, like CPU load, disk space, and network input. You can read these numbers directly using the Azure CLI without installing third-party agents. This gives you a fast snapshot of resource health when troubleshooting immediate performance spikes. Check metric names first before querying to ensure exact spelling.

```
$az monitor metrics list \
   --resource "/subscriptions/<SUB_ID>/resourceGroups/MyRG/providers/Microsoft.Compute/virtualMachine
   --metric "Percentage CPU" \
   --interval PT5M
```

Azure Monitor & Alerts: Know When Something Breaks Before Users Do

Sunday, October 25, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Querying Crash Logs with KQL

Azure Log Analytics stores plain text logs, error traces, and request events from all connected services. You query this data using Kusto Query Language (KQL), which works like a streamlined, readable version of SQL. It allows you to filter millions of log lines down to recent failure codes in seconds. Pipe operations step-by-step from data source to filter to presentation.

```
$az monitor log-analytics query \
  --workspace "<WORKSPACE_ID>" \
  --analytics-query "AppServiceHTTPLogs | where ScStatus >= 500 | summarize FailureCount=count() by
```

Azure Monitor & Alerts: Know When Something Breaks Before Users Do

Sunday, October 25, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Creating Automated Threshold Alerts

Metric alerts watch telemetry continuously and fire actions when conditions cross your safety limits. You define the metric, the evaluation frequency, and how high or low the value must go to trigger an alert. Always link your alert to an Action Group so notifications actually get sent out. Testing thresholds during quiet hours helps you tune out unnecessary noise.

```
$az monitor metrics alert create \
   --name "Alert-HighMemory" \
   --resource-group "MyRG" \
   --scopes "/subscriptions/<SUB_ID>/resourceGroups/MyRG/providers/Microsoft.Compute/virtualMachines/
   --condition "avg Available Memory Bytes < 1073741824" \
   --window-size 5m \
   --evaluation-frequency 1m \
   --action "/subscriptions/<SUB_ID>/resourceGroups/MyRG/providers/microsoft.insights/actionGroups/De
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Auditing Changes via Activity Log

The Activity Log records control-plane events across your entire subscription without requiring any setup. It answers the critical operational questions: who ran an action, what resource was modified, and what was the status? When an outage happens after a deployment, reviewing the Activity Log is your very first step to isolate recent manual changes.

```
$az monitor activity-log list \
   --resource-group "MyRG" \
   --start-time "2025-01-01T00:00:00Z" \
   --select caller eventTimestamp operationName status \
   --output table
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always configure at least one Action Group with verified team emails before creating alerts.
+ Use 5-minute evaluation windows for metric alerts to prevent temporary spikes from causing false alarms.
+ Group all related application logs into a single centralized Log Analytics workspace for easier querying.
+ Tag critical alert rules by environment name (like 'prod' or 'staging') so engineers know severity instantly.
+ Check Azure Service Health whenever multiple unrelated resources start failing at the exact same moment.

Watch Out For

! Setting threshold limits too strictly, leading to hundreds of emails and serious alert fatigue.
! Forgetting to route alerts to an Action Group, meaning alerts fire quietly with nobody listening.
! Querying massive time ranges (like 30 days) in Log Analytics during an incident, which slows down queries.
! Relying only on metric dashboards while completely forgetting to capture application-level error logs.

Follow for daily Cloud & DevOps guides!
linkedin.com/in/mramanrajsingh